PROJECT SYNOPSIS

LOCAL JOB FINDER — Cross-Platform Mobile Application

Domain:	Mobile App Development	**Target Market:**	Indian Employment Sector
Framework:	Flutter / Dart	**State Management:**	Provider Pattern
Backend:	Node.js REST API	**Local Database:**	Hive & Shared Preferences

---

1. INTRODUCTION & CONTEXT

The **Local Job Finder** project is an academic mobile application initiative designed to address job discovery and employer listing management in localized Indian regions. Built using Google's Flutter framework, the application delivers a unified, high-performance cross-platform codebase that adapts smoothly across smartphones, tablets, and desktop devices.

2. PROBLEM STATEMENT & MOTIVATION

Localized job markets often suffer from fragmented communication channels between small/medium employers and regional job seekers. This project aims to bridge this gap by establishing an accessible, fast, and offline-resilient mobile portal. From an engineering perspective, it addresses the challenge of building a multi-role, CRUD-heavy application with reliable synchronization between local client-side persistence and remote REST servers.

3. SYSTEM OBJECTIVES

- **Dual User Role Support:** Provide distinct, intuitive user flows for both Job Seekers (searching, filtering, applying, saving) and Employers (posting, editing, deleting listings).
- **Clean Software Architecture:** Implement a strict separation of concerns using the Provider state management pattern to decouple UI widgets from underlying business logic and API service layers.
- **Offline-First Resilience:** Utilize Hive local key-value persistence to cache job listings and application states, ensuring usability even without an active internet connection.
- **Full RESTful CRUD Operations:** Communicate with a live Node.js REST API for real-time creation, retrieval, updates, and deletion of job records.

4. SYSTEM MODULES & FUNCTIONAL ARCHITECTURE

The system is organized into 10 key interface modules:

Module / Screen	Functional Purpose
1. Splash & Onboarding	Initial application landing, branding, and feature introduction walkthroughs.
2. Auth (Login & Sign Up)	User identity validation with client-side form validation and state initialization.
3. Home Dashboard	Dynamic job feed with responsive grids, keyword search, and multi-field filtering (category, city, job type).
4. Job Details	

Module / Screen	Functional Purpose
	Detailed job descriptions, company information, salary ranges, direct action buttons (Save / Apply).
5. Post / Edit Job	Employer management form supporting both initial publishing and real-time updating of job entries.
6. Saved & Applied Jobs	Personalized candidate dashboard tracking saved bookmarks and submitted applications via Hive caching.
7. Profile & Settings	User account preferences, saved resume parameters, and role-switching controls.

5. TECHNICAL STACK

- **Frontend Framework:** Flutter (Dart) — Responsive layout system across phone, tablet, and desktop breakpoints.
- **State Management:** Provider Pattern — Centralized reactive state handling for app logic and filtering.
- **Database & Storage:** Hive (NoSQL key-value store) for fast offline caching; SharedPreferences for user session storage.
- **Networking / Backend:** RESTful API integration using standard HTTP protocols with Node.js / Express backend server.

6. IMPLEMENTATION METHODOLOGY & TIMELINE

The project follows a structured 6-week developmental methodology:

1. **Week 1 (Planning & Wireframing):** Repository setup, environment configuration, UI/UX wireframing for 10 screens.
2. **Week 2 (UI Execution):** Static UI building for Splash, Authentication, and Home screens.
3. **Week 3 (State Integration):** Wiring Provider state, reactive search, dynamic filter execution.
4. **Week 4 (Persistence & CRUD):** Local Hive database setup; implementation of Save, Apply, and Employer job management.
5. **Week 5 (REST Networking):** Connecting Flutter frontend to live REST backend; handling latency, empty, and offline states.
6. **Week 6 (QA & Deliverables):** Responsive testing, code refactoring, README documentation, and demo video production.

7. CONCLUSION & EXPECTED DELIVERABLES

The primary deliverables for the project include a clean, well-documented Flutter source code repository with integrated REST server backend code, a detailed `README.md`, a demonstration video showcasing end-to-end flows, and individual reflection reports.